

Boston Housing Price Prediction using DNN (Linear Regression)

Step 1: Import Required Libraries

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import boston_housing
import numpy as np
```

Step 2: Load the Boston Housing Dataset

```
(X_train, y_train), (X_test, y_test) = boston_housing.load_data()
```

Step 3: Normalize the Data

```
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std
```

Step 4: Build the DNN Model

```
model = models.Sequential([
    layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dense(1)
])
```

Step 5: Compile the Model

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

Step 6: Train the Model

```
model.fit(X_train, y_train, epochs=100, batch_size=16, validation_split=0.2, verbose=1)
```

Step 7: Evaluate the Model on Test Data

```
test_loss, test_mae = model.evaluate(X_test, y_test)
print(f"\nTest MAE (Mean Absolute Error): {test_mae:.2f}")
```

Step 8: Predict Some Prices

```
predictions = model.predict(X_test[:5])
print("\nPredicted prices vs Actual prices:")
for i in range(5):
    print(f"Predicted: {predictions[i][0]:.2f}, Actual: {y_test[i]:.2f}")
```

IMDB Review Classification with Decoding - Full Code

Step 1: Import Required Libraries

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np
```

Step 2: Load IMDB Dataset

```
num_words = 10000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=num_words)
```

Step 3: Preprocess Data

```
maxlen = 500
X_train = pad_sequences(X_train, maxlen=maxlen)
X_test = pad_sequences(X_test, maxlen=maxlen)
```

Step 4: Build the Deep Neural Network Model

```
model = models.Sequential([
    layers.Embedding(input_dim=num_words, output_dim=32, input_length=maxlen),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

Step 5: Compile the Model

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

Step 6: Train the Model

```
model.fit(X_train, y_train, epochs=5, batch_size=128, validation_split=0.2)
```

Step 7: Evaluate the Model

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"\nTest Accuracy: {test_acc:.4f}\n")
```

Step 8: Decode and Display Sample Reviews

```
word_index = imdb.get_word_index()
reverse_word_index = {value + 3: key for key, value in word_index.items()}
reverse_word_index[0] = '<PAD>'
```

IMDB Review Classification with Decoding - Full Code

```
reverse_word_index[1] = '<START>'
reverse_word_index[2] = '<UNK>'
reverse_word_index[3] = '<UNUSED>'

def decode_review(encoded_review):
    return ' '.join([reverse_word_index.get(i, '?') for i in encoded_review])

num_samples = 5
predictions = model.predict(X_test[:num_samples])

for i in range(num_samples):
    print(f"\n--- Review {i+1} ---")
    print(decode_review(X_test[i]))
    print(f"Actual Label: {'Positive' if y_test[i] == 1 else 'Negative'}")
    print(f"Predicted Label: {'Positive' if predictions[i] >= 0.5 else 'Negative'}")
    print(f"(Confidence: {predictions[i][0]:.2f})")
```

Fashion MNIST Classifier - Code with Comments

Step 1: Import Required Libraries

```
import tensorflow as tf
from tensorflow.keras import layers, models # For building the neural network
import matplotlib.pyplot as plt           # For visualization
import numpy as np                        # For numerical operations
```

Step 2: Load the Fashion MNIST Dataset

```
fashion_mnist = tf.keras.datasets.fashion_mnist
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
```

Step 3: Normalize the Data

```
X_train = X_train / 255.0 # Normalize pixel values to [0, 1]
X_test = X_test / 255.0
```

Step 4: Build the Neural Network Model

```
model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)), # Flatten 2D image to 1D vector
    layers.Dense(128, activation='relu'), # Hidden layer with 128 neurons
    layers.Dense(64, activation='relu'), # Another hidden layer with 64 neurons
    layers.Dense(10, activation='softmax') # Output layer for 10 classes
])
```

Step 5: Compile the Model

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Step 6: Train the Model

```
model.fit(X_train, y_train, epochs=10, validation_split=0.1)
```

Step 7: Evaluate the Model

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")
```

Step 8: Make Predictions

```
predictions = model.predict(X_test)
```

Step 9: Visualize Predictions

Fashion MNIST Classifier - Code with Comments

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']  
  
plt.figure(figsize=(10,10))  
for i in range(9):  
    plt.subplot(3, 3, i+1)  
    plt.xticks([])  
    plt.yticks([])  
    plt.imshow(X_test[i], cmap='gray')  
    plt.xlabel(class_names[np.argmax(predictions[i])])  
plt.tight_layout()  
plt.show()
```